Smoke-Resistant 3D Indoor Tactical Mapping

PRODUCT REQUIREMENT DOCUMENT (PRD) — 24-HOUR BUILD

24-HR HACKATHON

TARGET USERS	PRIMARY HARDWARE	DATA LINK	OUTPUT ENGINE
Incident Command & SAR	**Raspberry Pi Wearable**	**USB Tether / Local Hotspot**	**Three.js 3D Mesh**

1. EXECUTIVE SUMMARY & SYSTEM OVERVIEW

During interior structural fires, optical cameras and thermal imagers suffer severe degradation from aerosol backscatter in dense smoke, while standard GPS signals fail entirely indoors. This system provides **real-time 3D indoor spatial reconstruction and firefighter localization** in zero-visibility environments without optical line-of-sight. A wearable Raspberry Pi carried by the firefighter executes unthrottled sub-second Wi-Fi scans against static beacon anchors placed outside or around the perimeter. The central server fuses **Path-Loss Attenuation (RSSI)** signals with **Pedestrian Dead Reckoning (PDR)** via a Particle Filter to generate a live 3D wall boundary mesh and track firefighter coordinates on an Incident Command dashboard.

2. SYSTEM ARCHITECTURE & NETWORK TOPOLOGY

```
[ Static Beacon Anchors ] [ Firefighter Wearable Rig ] (3–4 Hotspots / Routers / ESP32s) - Raspberry Pi + US
```

Zero-Router Hardware Topology: The system operates over **USB Gadget Tethering (`g_ether`)**. Plugging the Raspberry Pi into the central laptop via USB creates a local virtual Ethernet connection (`169.254.x.x`), leaving the Pi's onboard Wi-Fi (`wlan0`) 100% dedicated to active sub-second background BSSID scanning without network drops.

3. SOFTWARE MODULES & DATA FORMATS

Module 1: Raspberry Pi Telemetry Scanner (`pi_client/pi_scanner.py`)

Scans surrounding Wi-Fi beacons via Linux `nmcli` in a 500ms loop and relays telemetry to `ws://192.168.1.100:5000/ws/telemetry`:

{ "device_id": "FF_Pi_Rig_1", "timestamp": 1718000521.12, "pdr": { "step_detected": true, "step_length_m": 0.75, "heading_deg": 84.5 }, "wifi_scans": [{ "bssid": "aa:bb:cc:11:22:33", "rssi_dbm": -52 }, { "bssid": "aa:bb:cc:44:55:66", "rssi_dbm": -68 }] }

3. MATHEMATICAL CORE & SENSOR FUSION

Trilateration & Log-Distance Path Loss

> Signal distance estimation model: $d = 10^{(P_0 - RSSI) / (10 \cdot n)}$
> Where P_0 is reference power at 1m, *RSSI* is measured signal strength in dBm, and $n \approx 2.5$ is the indoor path-loss exponent.

- **Particle Filter Fusion:** Combines IMU step vectors from the firefighter's wearable device with spherical RSSI distance estimations to update particle weights and maintain a drift-free coordinate (X_{FF}, Y_{FF}, Z_{FF}).
- **RF Attenuation Tomography:** Ray paths where $RSSI_{measured} << RSSI_{expected}$ are flagged as dense structural walls or obstacles.
- **3D Mesh Generation:** Marching Cubes algorithm processes a 0.25m³ 3D voxel density grid to generate structural wall wireframes in Three.js.

4. 24-HOUR EXECUTION TIMELINE & MILESTONES

Time Window	Phase	Core Objectives & Deliverables
00:00 – 03:00	Infra & USB Tethering	Configure Pi USB Gadget Mode (`g_ether`). Verify local ping connectivity to main laptop (`192.168.1.100`). Spin up FastAPI WebSocket skeleton.
03:00 – 08:00	Data Pipeline & Scanners	Deploy 500ms `nmcli` scan loop on Raspberry Pi. Stream live JSON payloads over WebSockets. Calibrate static anchor (x,y,z) positions.
08:00 – 16:00	Sensor Fusion & Mesh Engine	Implement Particle Filter in Python. Build RF Path-Loss Attenuation engine. Generate 3D voxel grid and Marching Cubes structural wall mesh.
16:00 – 21:00	Three.js Dashboard & Voice	Build Three.js 3D spatial viewport. Render live avatar, breadcrumbs, and wall mesh. Connect Web Speech API for voice radio hazard pinning.
21:00 – 24:00	Integration & Demo Prep	Perform blindfolded hallway walk-through test. Rig judging table with static beacon anchors for interactive scoring.

5. TEAM ROLE ALLOCATION MATRIX

Role / Member	Primary Ownership	Deliverables
Member A (Backend Lead)	FastAPI, WebSockets, Particle Filter	Python WebSocket server, sensor fusion math, RF tomography, voxel density grid.
Member B (Pi & Hardware)	Raspberry Pi OS, USB Tethering, Scanners	`g_ether` USB setup, `pi_scanner.py` script, static beacon anchor setup.
Member C (Frontend Lead)	React, Three.js 3D Canvas, WebGL	Real-time 3D viewport, voxel mesh renderer, avatar trajectory visualization.
Member D (Voice & Pitch)	Network Routing, Voice AI, Live Demo	Web Speech API radio pinning, integration testing, slide deck & live demo rig.

6. HACKATHON LIVE DEMO SETUP

1. Place 3 static Wi-Fi beacons (phone hotspots / routers) around the judging room.
2. Connect the Raspberry Pi to the main laptop via USB cable and power it on.
3. Have a teammate walk through the room blindfolded while holding the Pi rig.
4. Show judges the live Three.js screen updating the firefighter's position, carving out cleared space, and generating a 3D structural outline in real time.